

RAPPORT DE PENTEST

chaire-paas.com

Luca MANSI — 26 mai 2026

Scope : Lecture seule — Aucune destruction

1. Resume executif

Le site chaire-paas.com (Chaire Prevention et Acces aux Soins — Universite Cote d'Azur + SKEMA) est en phase de test (version 1.0.0.Beta). L'audit a revele 2 vulnerabilites critiques, 3 moyennes et plusieurs informatives.

Les deux vulnerabilites critiques permettent :

- D'obtenir les credentials admin directement depuis le JavaScript public
- D'extraire l'integralite de la base de donnees de recherche (~300 GB) sans aucune authentication

Tableau de synthese

ID	Titre	Severite	CVSS
CRIT-01	Credentials hardcodes dans JS public	Critique	9.8
CRIT-02	API sans auth - 300 GB exposes	Critique	9.1
MOYEN-01	DoS /commune (500 systematique)	Moyen	5.3
MOYEN-02	Pas de rate limiting	Moyen	5.3
MOYEN-03	Schema Pydantic expose (422)	Moyen	4.3
FAIBLE-01	Absence headers securite HTTP	Faible	3.7
FAIBLE-02	Nginx sans try_files SPA	Faible	2.1
INFO-01	Versions exposees	Info	-

2. Perimetre & Stack technique

Element	Valeur
URL cible	https://chaire-paas.com
IP	82.97.33.155 (Gandi)
Frontend	Angular 18 SPA
Serveur	nginx/1.24.0 (Ubuntu)
Backend	FastAPI Python
API Base	https://chaire-paas.com/api1
Version app	1.0.0.Beta (ICIP 3.2.0.Beta)

3. CRIT-01 — Credentials admin hardcodes dans le JS public

Severite : Critique | CVSS 9.8 | CWE-798

Le fichier JavaScript chunk-BR4JAKCK.js est accessible publiquement sans authentification. Il contient en clair la logique d'authentification Angular qui compare simplement les identifiants cote client.

Code source extrait

```
login(i,e){
  i=="admin"&&e=="Emlait$iA$0610"
  ?(console.log("Login ok"),this.isLogged.set(!0))
  :console.log("Login ko")
}
```

Credentials extraits

Champ	Valeur
Login	admin
Mot de passe	Emlait\$iA\$0610

PoC

```
curl -s https://chaire-paas.com/chunk-BR4JAKCK.js | grep -o 'Emlait[^"]*'
```

Output: Emlait\$iA\$0610

Impact

- Acces complet a la zone admin sans aucune requete serveur
- Auth 100% client-side — contournable par lecture directe du JS
- Credentials lisibles par n'importe quel visiteur ou bot
- Reutilisation possible sur d'autres services (VPS, Git, email)

Remediation

1. Supprimer la logique d'auth du frontend immediatement
2. Implanter auth serveur JWT (POST /api/login -> token signe backend)
3. Changer le mot de passe sur tous les services

4. CRIT-02 — API backend sans authentication (300 GB exposes)

Severite : Critique | CVSS 9.1 | CWE-306

Tous les endpoints de l'API FastAPI repondent sans token, cookie, ni aucune authentication. L'integralite de la base de donnees de recherche est accessible directement par n'importe qui.

Endpoints exposes (sans auth)

Methode	Endpoint	Donnees
GET	/api1/versions	Stack versions
GET	/api1/find/{commune}	BDD INSEE complete
POST	/api1/apl/iris	APL 27 specialites x 7 ans
POST	/api1/apl/iris/csv	Export CSV APL
POST	/api1/sae/iris	Donnees hospit. 5 types x 10 ans
POST	/api1/sae/iris/csv	Export CSV SAE

PoC — Paris, generaliste (aucun token)

```
curl -s -X POST https://chaire-paas.com/api1/apl/iris \
-H "Content-Type: application/json" \
-d '{"code":"CC-75056","id":10,"bor":"Generaliste","time":30,
    "exp":-0.12,"hc":"HC","resolution":"iris","apl_type":"APL"}'
# Reponse : 2.8 MB — 992 IRIS x 7 annees avec donnees socio-economiques
```

Exemple de donnees extraites

```
IRIS 751010201 "Les Halles 1" :
  APL          : 91.69
  Population   : 2122
  Taux pauvrete : 18%
  Revenu median : 33 800 EUR
  Gini         : 0.471

CHU DE NICE HOPITAL SAINT ROCH :
  Passages urgences : 78 711
  ETP total         : 23.33
  Tension           : 3373 passages/ETP
```

Volume extractible estime

Type	Perimetre	Volume estime
APL	34 935 communes x 27 specialites x 7 ans	~270 GB
SAE	34 935 communes x 5 types x 10 ans	~33 GB
Total		~303 GB

Remediation

```
# FastAPI - middleware auth JWT
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer

security = HTTPBearer()

async def verify_token(token = Depends(security)):
    if not valid_token(token.credentials):
        raise HTTPException(status_code=401)

@app.post("/apl/{geo_type}")
async def get_apl(payload: AplRequest, _=Depends(verify_token)):
    ...
```

5. Autres vulnerabilites

MOYEN-01 — DoS applicatif sur /commune

CVSS 5.3 | CWE-703

POST /api1/apl/commune retourne 500 Internal Server Error sur 100% des requetes.

```
curl -X POST https://chaire-paas.com/apil/apl/commune [...]
# HTTP 500 "Internal Server Error" (systematique, tous codes, toutes specialites)
```

MOYEN-02 — Absence de rate limiting

CWE-770

```
for i in $(seq 1 50); do curl -s https://chaire-paas.com/apil/find/Paris & done
# 50 req en 213ms - aucun 429 - aucun blocage
```

MOYEN-03 — Schema Pydantic expose via 422

```
{"detail":[{"type":"missing","loc":["body","hc"],"msg":"Field required",...}]}
```

FAIBLE-01 — Absence headers securite HTTP

Absents : Strict-Transport-Security, Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy

```
# nginx.conf
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Content-Security-Policy "default-src 'self'" always;
```

FAIBLE-02 — Nginx sans try_files

```
location / { try_files $uri $uri/ /index.html; }
```

6. Points positifs

OK CORS correctement restreint a chaire-paas.com

OK Pas de SQLi (requetes parametrees)

OK Pas de SSRF exploitable

OK Pas de directory listing

OK Pas de fichiers sensibles exposes (.env, .git...)

OK Pas de subdomains dangereux

OK TLS valide sur le domaine

OK Pas de RCE detecte

7. Plan de remediation

Immédiat (avant mise en production)

1. Retirer les credentials du JavaScript frontend
2. Implementer une auth serveur JWT sur /api1/
3. Corriger le bug 500 sur /apl/commune et /sae/commune

Court terme

4. Ajouter rate limiting nginx (limit_req_zone)
5. Configurer les headers de securite HTTP
6. Configurer try_files pour la SPA Angular
7. Desactiver les details de validation FastAPI en prod